Team Name : Team Flyhigh

Team Leader Name : Harini V M

Problem Statement : Intelligent Candidate Discovery & Ranking — rank 100,000 candidates for the Senior AI Engineer JD the way a great recruiter would

Solution Overview

- **What is your proposed solution?**

  An evidence-first hybrid ranker. It reads each candidate's career-history prose against a hand-derived model of the JD, gates out impossible (honeypot) and unsubstantiated (keyword-stuffed) profiles, applies the JD's explicit disqualifiers, weighs behavioral availability as a multiplier, and refines a 1,500-candidate shortlist with TF-IDF semantic similarity. The full 100,000-candidate pool is ranked in ~94 seconds on a laptop CPU — no LLM calls, no GPU, no network at rank time.

- **What differentiates your approach from traditional candidate matching systems?**

  Traditional matchers rank by closeness to the JD in keyword or embedding space — exactly where this dataset's traps are placed: a stuffer's skill list is engineered to sit next to the JD in vector space. We invert the usual hybrid: explicit JD-derived evidence reasoning drives the score, and semantic similarity is only a 15% tie-breaking refinement on an already-vetted shortlist. Evidence is read from what candidates actually did (career descriptions), never from self-reported skill lists.

JD Understanding & Candidate Evaluation

- **What are the key requirements extracted from the JD?**

  Three must-haves: production embeddings/retrieval, a shipped ranking/recommendation/search system at real-user scale, and evaluation rigor (NDCG/MRR/MAP, offline-online correlation, A/B). Plus: strong Python, 5–9 years (ideal 6–8) in applied ML at product companies, Pune/Noida-preferred India location, sub-30-day notice preferred. Explicit disqualifiers we encode: research-only careers, consulting-only careers, CV/speech-primary without NLP/IR, under-12-month LangChain-only AI experience, non-coding architect roles, and title-chasing job hoppers.

- **How does your solution evaluate candidate fit beyond keyword matching?**

  Nine concept lexicons (retrieval, ranking, evaluation, LLM, NLP/IR, production, learning-to-rank, open-source, HR-tech) run over career-history prose only, weighted by recency and product-company context — plain-language builders who never say 'RAG' or 'Pinecone' still surface. A skill claim counts only when the prose corroborates it. Behavioral signals (last-active, recruiter response rate, interview completion, notice period, relocation, visa) gate availability multiplicatively: a perfect-on-paper candidate who ignores recruiters is down-weighted, exactly as the JD instructs.

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**

  A six-stage funnel: stream-parse the JSONL → title prescreen on the raw line (the JD's own non-engineering rule removes 68% of the pool) → honeypot consistency gates → evidence extraction + structural fit + availability scoring → top-1,500 shortlist → TF-IDF cosine refinement → top-100 with reasoning.

- **What models, algorithms, or heuristics are used?**

  Compiled-regex concept lexicons with literal-substring pre-triggers (profiled: 9 min → 94 s); scikit-learn TfidfVectorizer (1–2 grams) for shortlist similarity; rule-based gates and multipliers. No learned weights — no labels exist — so every weight traces to a JD sentence (docs/JD_MAPPING.md in the repo carries the full table).

- **How are multiple candidate signals combined into a final ranking?**

  score = (core evidence × 1.25 retrieval-AND-ranking conjunction bonus + supporting evidence) × experience-band trapezoid (plateau 5–9, peak 6–8) × JD penalty multipliers × availability multiplier, then a 15% TF-IDF blend on the shortlist. Ties break deterministically by candidate_id.

Explainability & Data Validation

- **How are ranking decisions explained?**

  Every top-100 row carries a reasoning string generated from that candidate's extracted facts: title, employer, years, the actual evidence phrases matched in their own work history ('collaborative filtering', 'NDCG', 'semantic search'), and honest caveats — notice period, staleness, location. Tone is rank-consistent by construction (top-15 vs mid vs tail phrasing).

- **How do you prevent hallucinations or unsupported justifications?**

  Reasoning is assembled, not free-generated: only profile fields and verbatim-matched phrases can appear, so every claim traces to the record. No LLM touches candidate data at any point.

- **How does your solution handle inconsistent, low-quality, or suspicious profiles?**

  Internal-consistency gates catch the honeypots: job duration contradicting its own dates (>12 months), claimed experience exceeding the dated career span, 'expert' skills with zero months of use — 65 profiles excluded (~ the stated ~80), calibrated so generator noise doesn't trip. Uncorroborated AI skill lists are penalized ×0.10. Result: zero honeypot flags in our top-100 even under tightened thresholds.

End-to-End Workflow

- **What is the complete workflow from JD input to ranked candidate output?**

  One command: python rank.py --candidates candidates.jsonl --out submission.csv. (1) The JD reading lives in ranker/jd_profile.py as lexicons, weights, and disqualifier patterns — each mapped to a JD sentence. (2) Stream-parse 100K candidates; prescreen titles before JSON parsing. (3) Honeypot gates. (4) Evidence + structural + availability scoring per candidate. (5) Shortlist 1,500; TF-IDF refinement vs the JD text. (6) Emit top-100 CSV with scores and fact-grounded reasoning. Output passes the official validator; a Streamlit sandbox (app.py) runs the identical modules on uploaded samples for small-scale reproduction.

System Architecture

score = (core evidence × conjunction bonus + support) × experience band × JD penalties × availability — 94 s end-to-end, CPU-only, no network

Results & Performance

- **What results or insights demonstrate ranking quality?**

  Top-100 profile: mean 6.3 years' experience (JD ideal 6–8), 92% India-based, senior ML/AI engineers at product companies (Zomato, Razorpay, Sarvam AI, Paytm, CRED, Swiggy...), zero honeypot flags, zero non-engineering titles, every reasoning grounded in the candidate's own text. Ten unit tests encode the JD's canonical traps — stuffer vs builder, plain-language Tier-5 vs buzzword-lister, services-only careers, stale availability — and all pass.

- **How does your solution meet the challenge's runtime and compute constraints?**

  94 s wall-clock for the full 100K pool (budget: 300 s), ~1.5 GB peak RAM (budget: 16 GB), CPU-only, zero network and zero hosted-LLM calls during ranking, no precomputation step at all. Getting there required real profiling: regex alternations dominated the first build (9 min); an exact-title raw-line prescreen plus literal-substring triggers before authoritative regexes brought it to 94 s.

Technologies Used

- **What technologies, frameworks, and tools were used and why?**

  Python 3.12 standard library (json, re, datetime, csv) for the streaming pipeline — chosen for auditability, zero-dependency reproducibility, and CPU speed. scikit-learn TfidfVectorizer for shortlist semantic refinement — the only similarity component the trap design permits to matter. pytest for trap regression tests; Streamlit for the sandbox; git with real iterative history. Deliberately excluded from the rank path: LLM APIs and embedding services — per-candidate LLM calls cannot meet the 5-minute/100K budget, which mirrors the production reality the organizers are testing for.

Submission Assets

- **Submission assets**

  GitHub repository: rank.py + ranker/ (5 modules: jd_profile, gates, features, scoring, reasoning) + tests + docs/JD_MAPPING.md + this deck. Ranked output: output/submission.csv — top-100, official validator reports 'Submission is valid.' Sandbox: Streamlit app (app.py) — upload ≤100 candidates, identical pipeline, ranked CSV download. submission_metadata.yaml at repo root mirrors the portal metadata. Reproduce command: python rank.py --candidates ./candidates.jsonl --out ./submission.csv (~94 s, CPU-only).

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU